Technical Refactoring Plan: LogHelper & AtomicReference Strategy

December 8, 2025

1 Executive Summary

The primary objective of this refactoring is to adopt an `AtomicReference` approach for the `LogHelper` utility. This strategy ensures explicit thread safety, enables clean testing via dependency injection without reflection, and maintains the singleton nature of resources in the production environment.

2 Refactoring Specifications

2.1 Member Refactoring

- **Loggers:** Refactor `logger`, `fatLogger`, `kafkaLoggerInfo`, and `kafkaLoggerError` to be `private static final AtomicReference<Logger>`.
  - *Initialization:* Initialize these immediately with standard `LogManager.getLogger(...)` calls.
- **ObjectMapper:** Change the mapper to a `private static final ObjectMapper`.
  - *Configuration:* Use a static initializer block to configure it once (e.g., set `FAIL_ON_EMPTY_BEANS = false`).
- **Timing Context:** Convert all `ThreadLocal<Long>` and `ThreadLocal<Level>` fields to `ThreadLocal<Deque<Long>>` and `ThreadLocal<Deque<Level>>`.
  - *Initialization:* Initialize these using `ArrayDeque::new` to support nested timing scopes.

2.2 Method Logic Updates

- **Logging Calls:** Update all internal logging invocations to dereference the logger using `loggerRef.get().log(...)`.
- **Start Methods:** Update `start...` methods to push the current timestamp (or level) onto the `ThreadLocal` stack.
- **End Methods:**
  - Use `pollLast()` (or `pop`) to retrieve the start time and calculate duration.
  - **Cleanup:** If the stack is empty after popping, explicitly call `ThreadLocal.remove()` to prevent memory leaks in thread pools.

3 Test Support Strategy

3.1 Injection & Setup

- **Setters:** Add package-private static setter methods (e.g., `setLoggerForTest(Logger)`) to update the `AtomicReference` holding the logger.
- **Reflection Removal:** Remove reflection-based setup in `LogHelperTest`. Use the new static setter methods to inject mocks instead.

3.2 Test Execution

- **State Reset:** Ensure `@Before` or `@After` methods reset `AtomicReferences` to their original real loggers (or new mocks) to ensure test isolation and avoid pollution.
- **Timing Tests:**
  - Instead of setting `Long` via reflection, use the public `start...` method to push values.
  - Alternatively, use a package-private helper (e.g., `pushStartTimeForTest`) to inject a specific start time without triggering side-effect logging.

4 Implementation Patterns

4.1 Logger Holders & Test Setters

Define loggers as atomic references to allow hot-swapping during tests while keeping them static final.

```
// Production definitions
private static final AtomicReference<Logger> loggerRef =
    new AtomicReference<>(LogManager.getLogger("MainLogger"));
private static final AtomicReference<Logger> fatLoggerRef =
    new AtomicReference<>(LogManager.getLogger("Detail Logger"));
private static final AtomicReference<Logger> kafkaInfoRef =
    new AtomicReference<>(LogManager.getLogger("KafkaLoggerInfo"));
private static final AtomicReference<Logger> kafkaErrorRef =
    new AtomicReference<>(LogManager.getLogger("KafkaLoggerError"));

// Package-private setters for testing
static void setLoggerForTest(Logger mock) { loggerRef.set(mock); }
static void setFatLoggerForTest(Logger mock) { fatLoggerRef.set(mock); }
static void setKafkaInfoForTest(Logger mock) { kafkaInfoRef.set(mock); }
static void setKafkaErrorForTest(Logger mock) { kafkaErrorRef.set(mock); }
```

4.2 Shared ObjectMapper

Configure the mapper once in a static block.

```
private static final ObjectMapper mapper;

static {
    mapper = new ObjectMapper();
    mapper.configure(SerializationConfig.Feature.FAIL_ON_EMPTY_BEANS, false);
    // Additional configuration
}
```

4.3 Nested Timing Logic (ThreadLocal Stacks)

Handle nested method calls by using a Deque rather than a single Long.

```
// Stack Initialization
private static final ThreadLocal<Deque<Long>> startTimeStack =
    ThreadLocal.withInitial(ArrayDeque::new);

public void startLogDEBUG(Object... args) {
    startTimeStack.get().addLast(System.currentTimeMillis());
    // Existing logging behavior...
}

public void endLogDEBUG(String message, Object returnValue) {
    Deque<Long> dq = startTimeStack.get();
    Long start = dq.pollLast();

    if (start == null) return; // Handle gracefully

    long timeTaken = System.currentTimeMillis() - start;

    // Memory leak prevention: Clean up ThreadLocal when stack is empty
```

```
    if (dq.isEmpty()) {
        startTimeStack.remove();
    }

    // Log using the atomic reference
    loggerRef.get().log(Level.DEBUG, structureLog("End method", leanContext));
}
```

5 Notes & Edge Cases

- **Async Flows:** Be aware that `ThreadLocal` stacks do not carry over to different threads automatically. Use MDC for cross-thread context if required.
- **Dynamic Loggers:** If per-class loggers are required later (e.g., `LogManager.getLogger(callerClass)`), this pattern can be adapted to keep an instance-level logger while still sharing the static `ObjectMapper` and `ThreadLocal` stacks.
- **Current State:** The current code uses fixed names, so the static shared loggers defined above are appropriate.